

Что такое агент?

Gigaschool course

Статический контент (FAQ)



Реактивный chatbot (pattern matching)



LLM-based assistant (контекст + память)



Tool-using agent (вызов функций)



Autonomous agent (планирование + действия)

Система	Это агент?	Почему?
ChatGPT	Нет	Реагирует, не планирует действия
ChatGPT + Code Interpreter	Да	Может выполнять многошаговые задачи
Google Search в Gemini	Да	Планирует поиск + выбирает источники
Автопилот в Tesla	Да	Воспринимает, планирует, действует
Простой бот FAQ	Нет	Только pattern matching

Определение

Russell & Norvig (классическое): Агент воспринимает окружение ... и действует ... (PEAS framework)

Anthropic: Агент = автономность + tool use

OpenAI: Агент = reasoning + acting цикл

Hugging Face: Агент = AI модель, которая рассуждает, планирует и выполняет действия для достижения целей

Пример запроса к агенту

Input:

"Забронируй рейс в Барселону дешевле \$500"

Рассуждение: "Нужна информация о доступных рейсах"

Действие: Вызов поиска авиабилетов API

Результат: Фильтрация по цене

Действие: Бронирование выбранного рейса

В чём разница?

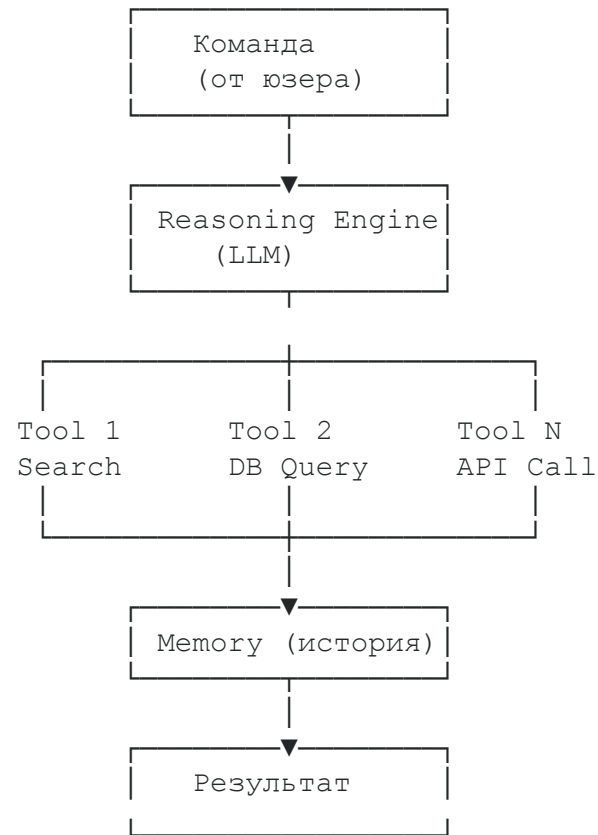
Chat-Bot:

- Принимает сообщение
- Генерирует ответ

Agent:

- Планирует
- Выполняет действия (функции)
- Проверяет результат

Агент — это цикл



Tool Use / Function Calling

Инструмент	Пример вызова	Результат
Web Search	<code>search("Apple stock price")</code>	Текущая цена акций
Database Query	<code>query_db("SELECT * FROM orders")</code>	Данные из БД
API Call	<code>call_api("weather", {"city": "Moscow"})</code>	Погода в Москве
Code Execution	<code>run_python("import math; math.sqrt(16)")</code>	4.0
Email	<code>send_email(to="user@mail.com", ...)</code>	Email отправлен

Memory

Short-term:

- Где: В контексте LLM (текущий разговор)
- Что: Последние 10-20 сообщений
- Проблема: Ограничена размером контекста
- Пример: Агент помнит, что 5 минут назад юзер попросил искать рейсы в Барселону

Long-term:

- Где: В базе данных, Vector Store
- Что: История всех взаимодействий, извлеченные знания
- Решение: Поиск по подобию (semantic search)
- Пример: "Я ранее искал рейсы туда — беру тот же результат"

Planning & Goal Decomposition

Задача: "Создай отчет о продажах за Q3"

Агент разбирает на подзадачи:

- Получить данные о продажах (→ API call)
- Агрегировать по категориям (→ Python execution)
- Создать визуализации (→ Chart generation)
- Написать анализ (→ LLM generation)
- Собрать PDF отчет (→ PDF export)

Structured Output (JSON)

Проблема:

LLM генерирует обычный текст

Решение: JSON

```
json
```

```
{  
  "reasoning": "Нужна информация о компании Apple",  
  "next_action": "search",  
  "action_input": {  
    "query": "Apple Inc. financial performance 2024"  
  }  
}
```

Structured Output (JSON)

Преимущества:

- Программа точно знает, что делать
- Валидация (проверка на ошибки)
- Легко парсить

Решение:

Инструменты:

- Pydantic (Python)
- TypeScript interfaces
- JSON Schema
- Zod (JavaScript)

Special Tokens и Fine-tuning

Special Tokens: Как модель "понимает", когда вызвать функцию?

Текст для модели:

```
"<tool>search</tool><input>Apple stock</input>"
```

Fine-tuning для Function Calling:

- Обучаем модель на примерах "правильного" использования инструментов
- Результат: агент точнее выбирает инструменты

Паттерны и архитектуры

ReAct (Reasoning + Acting)

Плюсы:

Просто и эффективно

Минусы:

Риск заикливания

1. Reason → "Мне нужна информация о ценах"
- ↓
2. Act → Вызвать search API
- ↓
3. Observe → Получить результат
- ↓
4. Repeat → Снова думать, нужны ли еще поиски?

Chain-of-Thought

Плюсы:

- Отладка
- Качество

Минусы:

- Скорость
- Стоимость

План действий:

- Поиск по компании Apple
- Получение данных о финансах
- Анализ трендов
- Генерация выводов

Reflection и Self-Critique

Плюсы:

- Автоисправление результатов
- Правдоподобие

Минусы:

- Скорость
- **Стоимость**

Концепция:

Выполни действие



Проверь результат



Переделай

Multi-Agent Debate

Плюсы:

- Ансамблирование
- Точность

Минусы:

- Скорость
- Стоимость

Концепция:

Агент А: "Я думаю, это хороший рейс за \$450"



Агент В: "Дороговато, есть \$399"



Агент С: "Я согласен с В"



Консенсус: Выбираем \$399

Сравнение паттернов

Паттерн	Сложность	Скорость	Когда
ReAct	Средняя	Быстро	Стандартные задачи
Chain-of-Thought	Средняя	Медленная	Сложные рассуждения
Reflection	Высокая	Очень медленно	Есть риск ошибки
Multi-Agent Debate	Очень высокая	Очень медленно	Research, анализ

Фреймворки от Hugging Face

3 основных фреймворка:

1. Smolagents

- Легко и быстро
- Для простых задач
- Максимум скорость

2. LlamaIndex

- Работа с данными
- Agentic RAG (поиск в документах)
- Для knowledge bases

3. LangGraph

- Для сложных workflow
- Когда много разветвлений

Agentic RAG

Традиционный RAG:

Вопрос → Поиск в БД → Генерация
(простой поиск)

Agentic RAG:

Вопрос → Агент решает как искать

- Поиск 1 (в договорах)
- Поиск 2 (в SLA)
- Поиск 3 (в FAQ)
 - └ Объединение результатов
 - └ Генерация ответа

Observability & Evaluation

1. Tracing

- Видим и проверяем запросы
- Инструменты: **Langfuse**, LangChain UI, LlamaIndex UI, Weights&Biases

2. Evaluation

- Метрики качества и бенчмарки
- Примеры метрик:
 - Accuracy (точность)
 - Latency (время ответа)
 - Cost (стоимость)

3. Leaderboard

- Сравнивать разные агентов
- Анализировать лучшие решения

Hallucination awareness

1. Strict schemas

- Явное ограничение списка инструментов

2. Validation

- Проверка использования инструмента

3. Fine-tuning

- Когда много денег и свободного времени

Другие проблемы и решения

Проблема	Причина	Решение
Infinite loops	Агент зациклился	Max steps limit, timeout
Ошибки инструментов	Плохой запрос к API	Error handling, retry logic
Дороговизна	Слишком много LLM запросов	Оптимизация промптов, caching
Медленность	Каждый шаг ждет LLM	Параллелизм, асинхронность
Отладка	Неясно, где ошибка	Логирование, trace UI

Безопасность и контроль

1. Логирование и алертинг

- Ставим уведомления на абьюз инструментов
- Ограничиваем применение инструментов в определённых сценариях

2. Валидация от человека

- Важные действия требуют подтверждения

3. Гардрейлы

- Проверяем аутпут агента на лики
- Собираем важную метаинформацию

Время вопросов